DayFlow

Productivity and Habit Tracker

Project Documentation

A full-stack web application for planning the day: tasks, habits, focus sessions, a daily productivity score and a weekly report. Built with React and Django REST Framework, and designed to be tested with Postman and Selenium WebDriver.

Item	Details
Version	1.0
Date	28 September 2026
Frontend	React 19 · Vite 8 · React Router 7 · Axios
Backend	Python 3.14 · Django 6.1 · Django REST Framework 3.18
Database	SQLite (default) · MySQL 8.4+ optional
Testing	Postman / Newman (included) · Selenium WebDriver + TestNG (planned, see section 9)

Contents

1. Project Description 5

1.1 How the parts fit together 5

1.2 Project goals 5

2. Features 6

2.1 Accounts and security 6

2.2 Tasks 6

2.3 Habits 6

2.4 Focus Mode 6

2.5 Productivity score 6

2.6 Dashboard, reports and profile 7

2.7 Quality and user experience 7

3. Technology Stack 8

3.1 Project folder structure 9

4. Installation 10

4.1 Required software 10

4.2 Check the installation 10

4.3 Allow PowerShell scripts (one time) 10

4.4 Get the project 10

4.5 Common installation problems 11

5. Database Setup 12

5.1 Default database: SQLite 12

5.2 Create the tables 12

5.3 Admin account (optional) 12

5.4 Data model 12

5.5 Optional: use MySQL instead of SQLite 13

5.6 Start again with an empty database (development only) 13

6. Backend Setup 14

6.1 Install and start the backend 14

6.2 Settings in backend/.env 14

6.3 Important backend behaviour 14

6.4 Check the backend 15

7. Frontend Setup 16

7.1 Install and start the frontend 16

7.2 Settings in frontend/.env . . . 16
7.3 npm commands . . . 16
7.4 Pages and routes . . . 16
7.5 How the frontend talks to the API . . . 17
8. API Documentation . . . 18
8.1 Basics . . . 18
8.2 Status codes . . . 18
8.3 Error format . . . 18
8.4 Authentication . . . 19
8.5 Tasks . . . 19
8.6 Habits . . . 20
8.7 Focus sessions . . . 21
8.8 Dashboard, productivity and weekly report . . . 21
9. Selenium Setup . . . 22
9.1 Tools . . . 22
9.2 Install Java and Maven . . . 22
9.3 Automation project structure . . . 22
9.4 pom.xml . . . 23
9.5 testng.xml . . . 23
9.6 Base test class . . . 24
9.7 Page Object and test example . . . 25
9.8 Locators (element ids) . . . 26
9.9 Tips for reliable tests . . . 27
10. Test Cases . . . 28
10.1 Authentication . . . 28
10.2 Access control and sessions . . . 29
10.3 Tasks . . . 29
10.4 Habits . . . 30
10.5 Focus Mode . . . 30
10.6 Dashboard and Reports . . . 31
10.7 Error handling . . . 31
10.8 API tests (Postman collection) . . . 31
11. How to Run the Application . . . 33
11.1 Terminal 1 — backend . . . 33
11.2 Terminal 2 — frontend . . . 33

11.3 Addresses . . . 33
11.4 First steps in the app . . . 33
11.5 Stop the application . . . 33
12. How to Run Automation Tests . . . 34
12.1 API tests in the Postman app . . . 34
12.2 API tests from the command line (Newman) . . . 34
12.3 Selenium UI tests (after section 9 is set up) . . . 34
12.4 Test data . . . 34
Appendix A. Known Limitations . . . 35

1. Project Description

DayFlow is a full-stack web application that helps a person plan the day and build good habits. After registering, the user manages daily tasks, ticks off habits, works in timed focus sessions, and follows a daily **productivity score** and a **weekly report**.

The project was built step by step as a learning project for full-stack development (React and Django REST Framework) and for test automation (Postman for the API, Selenium WebDriver with TestNG for the user interface). The interface is simple and responsive, and every important element has a stable **id**, so the application is easy to automate.

1.1 How the parts fit together



The React app never touches the database: every read and write goes through the API.

- The **React app** runs in the browser. It never connects to the database; it calls the Django REST API with JSON over HTTP.
- After logging in, every request carries the header `Authorization: Token <key>`.
- The **Django backend** owns all data. It checks every request, applies the business rules (for example the productivity score) and reads and writes the database through the Django ORM.
- The database is **SQLite** by default. MySQL can be switched on with a setting (section 5.5).

1.2 Project goals

- A real full-stack application with the frontend and backend kept separate and talking only through a REST API.
- Clean, readable, beginner-friendly code in a clear folder structure.
- An API that is easy to test with Postman (a ready-made collection is included).
- A user interface that is easy to automate with Selenium: stable ids, clear messages and predictable behaviour.

2. Features

2.1 Accounts and security

- Register with name, email, password and confirm password; log in with email and password (the email is not case-sensitive); log out.
- Passwords are stored hashed (PBKDF2) and must pass Django's password rules: at least 8 characters, not too common, not only numbers, not too similar to the email.
- Token authentication: the token is saved in the browser and sent with every request.
- Protected pages: logged-out users who open any page except Login and Register are sent to `/login`; logged-in users who open Login or Register are sent to the Dashboard.
- If the login stops working (for example after logging out on another device), the user is taken to the login page with the message "Your session has expired. Please log in again."
- Logging out in one browser tab logs out every open tab, and the Back button never shows a page from before the logout.

2.2 Tasks

- Add, edit and delete tasks (deleting asks for confirmation). A task has a title, description, priority (Low, Medium, High) and an optional due date.
- Tick a task to mark it completed (the completion time is recorded); untick to make it pending again.
- Search titles and descriptions, filter by priority and status, and page through the list (10 per page).
- Pending tasks that are past their due date show a red **Overdue** badge.
- Desktop shows a table; phones show cards.

2.3 Habits

- Daily habits, or weekly habits with a target of 1 to 6 days a week.
- Tick a habit to mark it done for today (untick to undo). A habit can be ticked once per day.
- Each habit shows its progress this week, for example "2 / 3 days", and a "Weekly goal met" badge when a weekly target is reached.
- A summary bar shows how many habits are done today.

2.4 Focus Mode

- Choose one of your pending tasks and a duration (1 to 180 minutes, 25 by default), then start the timer.
- The countdown can be paused and resumed. **Stop** cancels the session after a confirmation.
- At 00:00 the session is completed automatically and a "Focus Session Completed!" screen shows the duration and task.
- Refreshing the page or coming back later continues the running session.

2.5 Productivity score

- A score from 0 to 100 for each day: **50%** tasks (share of tasks due that day that are completed) + **30%** habits (share of habits done that day) + **20%** focus (1 point per completed focus minute, up to 100). The full formula is in section 8.8.

2.6 Dashboard, reports and profile

- Dashboard: greeting, four summary cards (tasks, habits, focus minutes, productivity), today's tasks with checkboxes, and weekly progress bars.
- Reports: a Monday-to-Sunday column chart of the daily score, a table of daily scores, the weekly average, the best day and today's score breakdown.
- Profile: name, email and the date the account was created.

2.7 Quality and user experience

- Friendly error messages everywhere; raw Django messages are never shown to users.
- Forms check required fields, email format and matching passwords before sending.
- Short confirmation messages ("toasts"), loading spinners, and "Try again" when data cannot be loaded.
- A friendly "Something went wrong" screen instead of a blank page if a page ever crashes.
- Responsive layout for desktop, tablet and phone; keyboard accessible.

3. Technology Stack

Layer	Technology	Version	Used for
Frontend	React	19.2	User interface (pages and components)
	Vite	8.3	Development server and production build
	JavaScript, HTML, CSS	—	Application code and styling (no CSS framework)
	React Router DOM	7.18	Page addresses and protected routes
	Axios	1.20	HTTP requests to the Django API
	oxlint	1.81	Code checking (npm run lint)
Backend	Python	3.14	Backend language
	Django	6.1.1	Web framework, database models, admin panel
	Django REST Framework	3.18.1	REST API, token authentication, pagination
	django-cors-headers	4.9.0	Allows the React app (port 5173) to call the API
	python-dotenv	1.2.3	Reads settings and secrets from the .env file
	mysqlclient	2.3.0	MySQL driver (only needed if MySQL is used)
Database	SQLite	3	Default database (a single file)
	MySQL	8.4 or newer	Optional database
API testing	Postman / Newman	Newman 6	API test collection (included in the project)
UI automation	Java JDK	21	Language for the Selenium tests (planned)
	Maven	3.9	Build tool and test runner (planned)
	Selenium WebDriver	4.49.0	Browser automation (planned)
	TestNG	7.12.0	Test framework (planned)
	Google Chrome	current	Browser used by the tests
Tools	Node.js / npm	24 / 11	Frontend tooling
	VS Code, Git	—	Editor and version control

3.1 Project folder structure

```
DayFlow/
├── backend/                    Django REST API
│   ├── manage.py
│   ├── requirements.txt        Python packages
│   ├── .env / .env.example     settings and secrets (.env is never committed)
│   ├── db.sqlite3              SQLite database (created by "migrate")
│   ├── config/                 settings.py, urls.py, pagination.py
│   ├── users/                  User model; register, login, logout, current user
│   ├── tasks/                  Task model; CRUD, search, filter, sort
│   ├── habits/                 Habit and HabitCompletion; "complete for today"
│   ├── focus/                  FocusSession; start, complete, cancel, history, active
│   └── dashboard/              productivity score, dashboard numbers, weekly report
├── frontend/                   React app
│   ├── package.json, .env, index.html, vite.config.js
│   └── src/
│       ├── main.jsx            starts the app (router, login state, toasts)
│       ├── App.jsx             all routes and the route guards
│       ├── components/         Button, Input, Modal, TaskTable, HabitCard, FocusTimer, ...
│       ├── pages/              Login, Register, Dashboard, Tasks, Habits, Focus, Reports, Profile, 404
│       ├── services/           api.js (axios), one file per API area, errors.js
│       ├── context/            login state (AuthProvider) and toasts (ToastProvider)
│       └── utils/              formatting, form checks, screen-size hook
├── postman/
│   └── DayFlow.postman_collection.json   API tests (52 requests, 106 checks)
├── docs/
│   └── DayFlow_Project_Documentation.pdf this document
└── automation/                 Selenium + TestNG project (planned, section 9)
```

4. Installation

This section installs the software DayFlow needs and checks that it works. The commands are for **Windows PowerShell**, the terminal used during development.

4.1 Required software

Software	Version used	Needed for	Check with
Python	3.14	Backend	`python --version`
pip	26	Installing Python packages	`pip --version`
Node.js	24	Frontend	`node --version`
npm	11	Installing frontend packages	`npm --version`
Google Chrome	current	Using the app; Selenium tests	`open chrome://version`
VS Code	current	Editing code (recommended)	`code --version`
Git	current	Version control (recommended)	`git --version`
Postman	current	API testing	`open the Postman app`
Java JDK	21	Selenium tests (planned)	`java -version`
Maven	3.9	Selenium tests (planned)	`mvn -version`
MySQL	8.4+	Only if MySQL is used instead of SQLite	`mysql --version`

4.2 Check the installation

Open a new PowerShell window and run:

```
python --version
pip --version
node --version
npm --version
```

Each command should print a version number. The Java, Maven and MySQL checks are only needed for the Selenium tests and for MySQL.

4.3 Allow PowerShell scripts (one time)

Windows blocks PowerShell scripts by default, which stops `npm`, `npx` and the Python virtual environment from working. Allow scripts for your user once:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

4.4 Get the project

Copy (or clone) the `DayFlow` folder to your computer. All commands in this document start from that folder, for example `C:\Users\Admin\Desktop\Car-Game\DayFlow`. Then continue with the database, backend and frontend setup (sections 5 to 7).

4.5 Common installation problems

Problem	Cause	Fix
"X is not recognized as the name of a cmdlet"	Not installed, or its folder is not on PATH	Install it or add its folder to PATH, then restart VS Code / the terminal
"python" opens the Microsoft Store	Windows app alias	Settings → Apps → Advanced app settings → App execution aliases → turn off python.exe and python3.exe
"npm.ps1 cannot be loaded because running scripts is disabled"	PowerShell script policy	Run the command in 4.3
New PATH entries not found in VS Code	VS Code keeps the PATH it started with	Close VS Code completely and reopen it
"pip" not recognized but python works	pip folder not on PATH	Use `python -m pip ...` instead

5. Database Setup

5.1 Default database: SQLite

DayFlow uses **SQLite** by default. SQLite keeps the whole database in one file, `backend/db.sqlite3`, and is built into Python, so there is nothing to install. The database file and tables are created by the `migrate` command.

5.2 Create the tables

With the backend virtual environment active (see section 6):

```
cd DayFlow\backend
.\venv\Scripts\Activate.ps1
python manage.py migrate
```

`migrate` runs every migration file and creates the tables: users, tasks, habits, habit completions, focus sessions, login tokens and Django's own tables (sessions, admin log, permissions).

5.3 Admin account (optional)

Django includes an admin panel for viewing and editing all data. Create an admin account; it asks for **Email**, **Name** and **Password**:

```
python manage.py createsuperuser
```

Then log in at `http://127.0.0.1:8000/admin/` while the backend is running.

5.4 Data model

Table (model)	Main fields	Rules
User	id, name, email, password, is_active, is_staff, date_joined	Email is unique and is the login name (stored in lowercase). Password is hashed.
Task	id, user, title, description, priority, status, due_date, created_at, updated_at, completed_at	Title up to 200 characters. Priority low / medium / high (default medium). Status pending / completed. completed_at is set automatically.
Habit	id, user, name, description, frequency, target_days, created_at	Name up to 100 characters. Frequency daily / weekly. target_days is 7 for daily habits and 1–6 for weekly habits.
HabitCompletion	id, habit, date, completed, created_at	At most one row per habit per day. completed = false means the tick was undone.
FocusSession	id, user, task, duration, start_time, end_time, status	Duration 1–180 minutes. Status running / completed / cancelled. One running session per user. If the task is deleted, the session stays in history with no task.
Token	key, user, created	Login token (Django REST Framework). Deleted on logout.

Every task, habit and focus session belongs to one user. The API only ever returns the logged-in user's own data.

5.5 Optional: use MySQL instead of SQLite

> **Version requirement:** Django 6.1 needs **MySQL 8.4 or newer** (or MariaDB 10.11+). Older versions such as MySQL 8.0 or the MariaDB 10.4 that comes with XAMPP are refused by Django. Either upgrade MySQL, or use Django 5.2 LTS, which supports MySQL 8.0.

1. Create the database in MySQL:

```
CREATE DATABASE dayflow CHARACTER SET utf8mb4;
```

2. In `backend/.env`, switch the engine and enter the connection details:

```
DB_ENGINE=mysql
DB_NAME=dayflow
DB_USER=root
DB_PASSWORD=your_password
DB_HOST=127.0.0.1
DB_PORT=3306
```

3. Run `python manage.py migrate` again to create the tables in MySQL. The database password stays in .env and is never written in the source code.

5.6 Start again with an empty database (development only)

> This permanently deletes all users, tasks, habits and focus sessions.

```
# stop the backend first (Ctrl+C)
Remove-Item db.sqlite3
python manage.py migrate
```

6. Backend Setup

6.1 Install and start the backend

```
cd DayFlow\backend
python -m venv venv                  # create a private Python environment (first time only)
.\venv\Scripts\Activate.ps1          # switch to it; the prompt starts with (venv)
pip install -r requirements.txt      # install Django, DRF and the other packages
Copy-Item .env.example .env          # create your settings file (first time only)
python manage.py migrate             # create the database tables
python manage.py runserver           # start the API at http://127.0.0.1:8000
```

> **Always activate the virtual environment** before running backend commands. If `python -m django --version` does not print 6.1.1, the venv is not active.

6.2 Settings in backend/.env

Open `.env` and set a new secret key. You can generate one with `python -c "import secrets; print(secrets.token_urlsafe(50))"`.

Variable	Example	Meaning
SECRET_KEY	a long random text	Django's secret key. Keep it private.
DEBUG	True	Detailed error pages for development. Use False in production.
ALLOWED_HOSTS	127.0.0.1,localhost	Host names the backend answers to.
CORS_ALLOWED_ORIGINS	http://localhost:5173,http://127.0.0.1:5173	Frontend addresses allowed to call the API.
DB_ENGINE	sqlite	sqlite (default) or mysql.
DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT	dayflow, root, …	MySQL connection (only used when DB_ENGINE=mysql).

6.3 Important backend behaviour

Setting	Value	Why
Time zone	Asia/Kolkata	"Today" for tasks, habits and the score follows Indian time.
Authentication	Token only	Clients send "Authorization: Token <key>".
Default permission	Logged-in users only	Only register and login are public.
Pagination	10 per page, up to 100 with ?page_size	Lists never return unlimited data.
Password rules	Django's 4 standard validators	Minimum length, common, numeric and similarity checks.

6.4 Check the backend

```
python manage.py check            # should print: System check identified no issues (0 silenced).
```

Open `http://127.0.0.1:8000/admin/`: the Django admin login page means the backend is running.

7. Frontend Setup

7.1 Install and start the frontend

```
cd DayFlow\frontend
npm install          # download React, Vite, React Router and Axios (first time, or after changes)
npm run dev          # start the app at http://localhost:5173
```

7.2 Settings in frontend/.env

```
VITE_API_URL=http://127.0.0.1:8000/api
```

This is the address of the Django API. Restart the dev server after changing it.

7.3 npm commands

Command	What it does
`npm run dev`	Development server with instant reload at http://localhost:5173
`npm run build`	Optimised production build in the dist/ folder
`npm run preview`	Serves the production build locally for a final check
`npm run lint`	Checks the code with oxlint

7.4 Pages and routes

Route	Page	Access
`/login`	Login	Logged-out users (logged-in users go to /dashboard)
`/register`	Register	Logged-out users (logged-in users go to /dashboard)
`/dashboard`	Dashboard	Logged-in users
`/tasks`	Tasks	Logged-in users
`/habits`	Habits	Logged-in users
`/focus`	Focus Mode	Logged-in users
`/reports`	Reports	Logged-in users
`/profile`	Profile	Logged-in users
`/`	—	Goes to /dashboard (and to /login when logged out)
`any other address`	404 page	Everyone

All routes are defined in `src/App.jsx`. Logged-out users who open a protected page are redirected to `/login`; after logging in they go to `/dashboard`.

7.5 How the frontend talks to the API

- `src/services/api.js` creates one Axios client with the API address from .env.
- The login token is saved in the browser's localStorage under the key `dayflow_token`, and Axios adds `Authorization: Token <key>` to every request.
- If the API answers 401 (login no longer valid), the token is removed and the user is sent to the login page with a "session expired" message.
- `src/services/errors.js` turns every API error into a friendly message before it is shown.

8. API Documentation

8.1 Basics

Item	Details
Base URL	`http://127.0.0.1:8000/api`
Format	JSON requests and responses. Send the header Content-Type: application/json.
Authentication	`Authorization: Token <key>` on every endpoint except register and login. The 40-character key comes from register or login.
Trailing slash	Required: `/api/tasks/`, not `/api/tasks`.
Dates and times	Dates as `YYYY-MM-DD`; times in ISO 8601 with the +05:30 time zone.
Choice values	Lowercase: low / medium / high, pending / completed, daily / weekly, running / completed / cancelled.
Lists	Paginated: `{"count", "next", "previous", "results"}`, 10 per page. Use `?page=2` and `?page_size=50` (maximum 100).
Own data only	Every list and detail endpoint only sees the logged-in user's data. Another user's id returns 404.

8.2 Status codes

Code	Meaning in DayFlow
200 OK	Request succeeded
201 Created	Account, task, habit or focus session created
204 No Content	Deleted, or no running focus session (GET /focus/active/)
400 Bad Request	Invalid data; the body says which field and why
401 Unauthorized	Missing, invalid or logged-out token
404 Not Found	Item does not exist or belongs to another user; or the page number is too high
405 Method Not Allowed	Wrong HTTP method for the endpoint
500 Server Error	Unexpected problem on the server

8.3 Error format

```
# A problem with one field: the field name, then its messages
{ "title": ["This field may not be blank."] }

# A problem with the request as a whole
{ "non_field_errors": ["Invalid email or password."] }

# A rule that stopped the action (also used for 401 and 404)
{ "detail": "You already have a focus session running. Complete or cancel it first." }
```

The API keeps Django's wording (the Postman tests check these exact texts). The React app translates them into friendlier messages for users, for example "Title is required."

8.4 Authentication

Method	Endpoint	Body	Success response
POST	/auth/register/	name, email, password, confirm_password	201 {"token", "user"}
POST	/auth/login/	email, password	200 {"token", "user"}
POST	/auth/logout/	—	200 {"detail": "Logged out successfully."}
GET	/auth/me/	—	200 {"id", "name", "email", "date_joined"}

Example: register

```
POST /api/auth/register/
{
  "name": "Asha",
  "email": "asha@example.com",
  "password": "Sunrise#2026",
  "confirm_password": "Sunrise#2026"
}

201 Created
{
  "token": "d9203f7ff592ef0ca5972ce15cd0a3e8659affd1",
  "user": { "id": 1, "name": "Asha", "email": "asha@example.com",
            "date_joined": "2026-09-28T11:11:01+05:30" }
}
```

Situation	Response
Email already used	400 {"email": ["An account with this email already exists."]}
Passwords differ	400 {"confirm_password": ["Passwords do not match."]}
Weak password	400 {"password": ["This password is too short. It must contain at least 8 characters.", ...]}
Wrong email or password	400 {"non_field_errors": ["Invalid email or password."]}
No token / bad token	401 {"detail": "Authentication credentials were not provided."} / {"detail": "Invalid token."}

One token is kept per user and reused on every login. Logging out deletes it, which ends the login on all devices.

8.5 Tasks

Method	Endpoint	Description	Success
GET	/tasks/	List your tasks (search, filter, sort, pages)	200
POST	/tasks/	Create a task	201
GET	/tasks/{id}/	Get one task	200
PUT	/tasks/{id}/	Update a task; title required, fields left out keep their values	200
PATCH	/tasks/{id}/	Update only the fields sent, e.g. {"status": "completed"}	200
DELETE	/tasks/{id}/	Delete a task	204

Query parameter	Example	Meaning
search	?search=java	Title or description contains the text (not case-sensitive)
status	?status=pending	pending or completed

Query parameter	Example	Meaning
priority	?priority=high	low, medium or high
due_date	?due_date=2026-09-28	Tasks due on that date
ordering	?ordering=-priority	created_at, due_date, priority or title; "-" = descending. Default -created_at
page, page_size	?page=2&page_size=20	Pagination

Example: create a task

```
POST /api/tasks/        { "title": "Learn Java", "description": "Selenium needs Java",
                          "priority": "high", "due_date": "2026-09-28" }

201 Created
{ "id": 1, "title": "Learn Java", "description": "Selenium needs Java", "priority": "high",
  "status": "pending", "due_date": "2026-09-28", "created_at": "...", "updated_at": "...",
  "completed_at": null }
```

Only **title** is required. Defaults: priority medium, status pending, no due date. Setting status to completed fills completed_at; setting it back to pending clears it. Invalid filter values return 400, for example `{"status": ["Must be one of: pending, completed."]}`.

8.6 Habits

Method	Endpoint	Description	Success
GET	/habits/	List your habits, oldest first	200
POST	/habits/	Create a habit	201
GET	/habits/{id}/	Get one habit	200
PUT / PATCH	/habits/{id}/	Update a habit	200
DELETE	/habits/{id}/	Delete a habit and its history	204
POST	/habits/{id}/complete/	Tick for today: {"completed": true} (default) or untick: {"completed": false}	200

Habit object

```
{ "id": 3, "name": "Exercise", "description": "", "frequency": "weekly", "target_days": 3,
  "created_at": "...", "completed_today": true, "completions_this_week": 1 }
```

- Daily habits always have target_days 7. Weekly habits need target_days from 1 to 6, otherwise: 400 "For weekly habits, choose 1 to 6 days per week. Use "daily" for every day."
- Only today can be ticked. Ticking twice still counts once. Weeks run Monday to Sunday.
- completed_today and completions_this_week are calculated by the server.

8.7 Focus sessions

Method	Endpoint	Description	Success
POST	/focus/start/	Start a session: {"task": <your task id>, "duration": 25}	201
POST	/focus/complete/	Complete the running session (only when its time is up)	200
POST	/focus/cancel/	Cancel the running session	200
GET	/focus/history/	All sessions, newest first (paginated)	200
GET	/focus/active/	The running session, or 204 when none	200 / 204

Focus session object

```
{ "id": 1, "task": 31, "task_title": "Learn Java", "duration": 25, "status": "running",
  "start_time": "...", "planned_end_time": "...", "end_time": null, "remaining_seconds": 1475 }
```

- duration is in minutes, 1 to 180 (default 25). task is required and must be one of your own tasks.
- Only one session can run at a time: 400 "You already have a focus session running. Complete or cancel it first."
- Completing too early: 400 "This session has N minute(s) left. Cancel it to stop early." (a 5-second allowance covers network delay).
- Complete or cancel with nothing running: 400 "You have no focus session running."

8.8 Dashboard, productivity and weekly report

Method	Endpoint	Response
GET	/dashboard/	{"total_tasks", "completed_tasks", "pending_tasks", "total_habits", "completed_habits", "productivity_score", "focus_minutes"}
GET	/dashboard/productivity/	{"task_completion", "habit_completion", "focus_score", "productivity_score"}
GET	/dashboard/weekly-report/	[{"date": "Monday", "score": 80}, ...] from Monday to today

Productivity score (0–100) for a day:

```
task_completion    = % of tasks due that day that are completed        (0 if none are due)
habit_completion   = % of habits done that day                           (0 if there are no habits)
                     a weekly habit also counts once its weekly target is met
focus_score        = minutes of focus completed that day, up to 100

productivity_score = 0.5 x task_completion + 0.3 x habit_completion + 0.2 x focus_score
                     rounded to a whole number, .5 rounds up

Example: 75% tasks, 50% habits, 75 focus  ->  37.5 + 15 + 15 = 67.5  ->  68
```

On the dashboard, the task numbers count all tasks; habits, focus minutes and the score are for today. A day in the weekly report is scored the same way, using the tasks due and habits done on that day.

9. Selenium Setup

> **Status: planned.** The Selenium automation project is not part of the codebase yet. This section describes how to set it up when automation starts. The locators and messages listed here are the ones the application uses today.

9.1 Tools

Tool	Version	Notes
Java JDK	21 (LTS)	Eclipse Temurin recommended
Maven	3.9	Downloads Selenium and TestNG, runs the tests
Selenium WebDriver (selenium-java)	4.49.0	Includes Selenium Manager, which downloads the matching ChromeDriver automatically; no separate driver download is needed
TestNG	7.12.0	Test framework: annotations, assertions, suites, reports
Maven Surefire plugin	3.6.0	Runs the TestNG suite with mvn test
Google Chrome	current	The browser the tests control

9.2 Install Java and Maven

```
winget install --id EclipseAdoptium.Temurin.21.JDK -e       # Java 21 (asks for admin permission)

# Set JAVA_HOME for your user
$adoptium = "C:\Program Files\Eclipse Adoptium"
$jdk = (Get-ChildItem $adoptium -Directory -Filter "jdk-21*" | Select-Object -First 1).FullName
[Environment]::SetEnvironmentVariable("JAVA_HOME", $jdk, "User")
```

For Maven, download the **Binary zip archive** from maven.apache.org/download.cgi, extract it to `C:\tools\`, and add its `bin` folder to your user PATH. Then close and reopen VS Code and check:

```
java -version        # should show version 21
mvn -version         # should show Apache Maven 3.9.x and Java 21
```

9.3 Automation project structure

```
automation/
├── pom.xml                          dependencies and the test runner
├── testng.xml                       which test classes to run
└── src/test/java/com/dayflow/
    ├── base/BaseTest.java           opens and closes Chrome for every test
    ├── pages/                       Page Objects: LoginPage, RegisterPage, DashboardPage,
    │                                TasksPage, HabitsPage, FocusPage, ReportsPage
    ├── utils/                       test data helpers (unique emails, API calls)
    └── tests/                       AuthTests, AccessTests, TaskTests, HabitTests,
                                     FocusTests, DashboardTests, ReportTests, ErrorTests
```

Tests use the **Page Object Model**: each page gets one class holding its locators and actions, so a change in the UI is fixed in one place instead of in every test.

9.4 pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.dayflow</groupId>
  <artifactId>dayflow-automation</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.49.0</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.12.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.6.0</version>
        <configuration>
          <suiteXmlFiles>
            <suiteXmlFile>testng.xml</suiteXmlFile>
          </suiteXmlFiles>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

9.5 testng.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="DayFlow UI Tests">
  <test name="Authentication">
    <classes>
      <class name="com.dayflow.tests.AuthTests"/>
    </classes>
  </test>
</suite>
```

9.6 Base test class

```
package com.dayflow.base;

import java.time.Duration;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

public class BaseTest {
    protected static final String BASE_URL = "http://localhost:5173";
    protected WebDriver driver;
    protected WebDriverWait wait;

    @BeforeMethod
    public void setUp() {
        ChromeOptions options = new ChromeOptions();
        if (Boolean.getBoolean("headless")) {          // mvn test -Dheadless=true
            options.addArguments("--headless=new");
        }
        options.addArguments("--window-size=1280,900");
        driver = new ChromeDriver(options);             // Selenium Manager finds ChromeDriver
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

9.7 Page Object and test example

src/test/java/com/dayflow/pages/LoginPage.java

```
package com.dayflow.pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class LoginPage {
    private final WebDriver driver;
    private final WebDriverWait wait;
    private final By email = By.id("login-email");
    private final By password = By.id("login-password");
    private final By submit = By.id("login-submit");
    private final By error = By.id("login-error");

    public LoginPage(WebDriver driver, WebDriverWait wait) {
        this.driver = driver;
        this.wait = wait;
    }

    public void open(String baseUrl) {
        driver.get(baseUrl + "/login");
    }

    public void login(String userEmail, String userPassword) {
        wait.until(ExpectedConditions.visibilityOfElementLocated(email)).sendKeys(userEmail);
        driver.findElement(password).sendKeys(userPassword);
        driver.findElement(submit).click();
    }

    public String errorMessage() {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(error)).getText();
    }
}
```

src/test/java/com/dayflow/tests/AuthTests.java

```
package com.dayflow.tests;

import com.dayflow.base.BaseTest;
import com.dayflow.pages.LoginPage;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.testng.Assert;
import org.testng.annotations.Test;

public class AuthTests extends BaseTest {

    @Test
    public void invalidLoginShowsFriendlyMessage() {
        LoginPage loginPage = new LoginPage(driver, wait);
        loginPage.open(BASE_URL);
        loginPage.login("nobody@test.com", "WrongPassword1");
        Assert.assertEquals(loginPage.errorMessage(), "Invalid email or password.");
    }

    @Test
    public void loggedOutUserIsSentToLogin() {
        driver.get(BASE_URL + "/dashboard");
        wait.until(ExpectedConditions.urlToBe(BASE_URL + "/login"));
    }
}
```

9.8 Locators (element ids)

Every important element has an id; use `By.id(...)`. A field's error message uses the field id plus `-error`, for example `task-title-error`. Items from the database include their id, for example `task-12-checkbox`.

Page	Element ids
Login	`login-email, login-password, login-submit, login-error, login-info (session expired), register-link`
Register	`register-name, register-email, register-password, register-confirm-password, register-submit, register-error, login-link`
Layout	`nav-dashboard, nav-tasks, nav-habits, nav-focus, nav-reports, nav-profile, navbar-user-name, logout-button, menu-button (phones); toasts: #toast-container .toast-message`
Dashboard	`dashboard-title, dashboard-date, stat-tasks-value, stat-habits-value, stat-focus-value, stat-progress-value, today-tasks, today-task-<id>, today-tasks-empty, weekly-<day>-value, view-all-tasks, view-reports`
Tasks	`add-task-button, task-search, filter-priority, filter-status, task-count, task-table, task-<id>, task-<id>-title, task-<id>-checkbox, task-<id>-edit, task-<id>-delete, tasks-empty, clear-filters, pagination-prev, pagination-info, pagination-next`
Task modal	`task-modal, task-modal-title, task-title, task-description, task-priority, task-due-date, task-save, task-cancel, task-form-error`
Confirmation	`confirm-modal-message, confirm-modal-confirm, confirm-modal-cancel (Focus: stop-modal-...)`
Habits	`add-habit-button, habits-progress-value, habit-list, habit-<id>, habit-<id>-checkbox, habit-<id>-name, habit-<id>-progress-value, habit-<id>-edit, habit-<id>-delete; modal: habit-name, habit-description, habit-frequency, habit-target-days, habit-save, habit-cancel`
Focus	`focus-task, focus-duration, focus-start-button, focus-time, focus-task-title, focus-pause-button, focus-paused, focus-stop-button, focus-completed-title, focus-completed-duration, focus-completed-task, back-to-dashboard, start-another, focus-no-tasks`
Reports	`weekly-chart, chart-bar-<day> (attribute data-score), weekly-table, report-score-<day>, report-average-value, report-best-value, report-today-value`
Profile / other	`profile-name, profile-email, profile-joined, not-found-title, app-error-title, app-error-reload`

9.9 Tips for reliable tests

Situation	What to do
Pages load their data after they open	Always use explicit waits (WebDriverWait + ExpectedConditions); never Thread.sleep or findElement straight after navigation.
StaleElementReferenceException	Lists and toasts are re-drawn when data changes. Use wait.ignoring(StaleElementReferenceException.class) and find elements again.
clear() does not empty React inputs	Use sendKeys(Keys.CONTROL + "a") then Keys.BACKSPACE; for date inputs press Keys.BACKSPACE.
"element click intercepted" by the top bar	Scroll the element to the middle first: js.executeScript("arguments[0].scrollIntoView({block:'center'})", element).
Toasts cover buttons	Wait for the toast to disappear, or close it with its close (X) button.
Phone-size tests	Chrome windows cannot be narrower than 500 px; use Chrome device emulation. Click menu-button before using the sidebar links.
Registering twice fails	Use a unique email per run, e.g. "user" + System.currentTimeMillis() + "@test.com".
Slow setup through the UI	Create test data through the API first, then check it in the browser.
Focus timer	Use duration 1 and wait about 65 seconds for completion.
Date and time	The greeting and the "today" numbers depend on the clock; assert the user name, not the exact greeting.

10. Test Cases

The test cases below describe what DayFlow should do. The **expected results** quote the exact texts the application shows, so they can be used for manual testing and turned into Selenium tests. Before each test, start the backend and frontend (section 11).

Prefix	Area	Prefix	Area
AUTH	Register, login, logout	FOC	Focus Mode
SEC	Access control and sessions	DASH	Dashboard
TASK	Tasks	REP	Reports
HAB	Habits	ERR	Error handling

10.1 Authentication

ID	Test case	Steps	Expected result
AUTH-01	Register with valid data	Open /register; enter name, a new email, password Sunrise#2026 twice; click Create account	Dashboard opens; toast "Account created. Welcome to DayFlow!"; the name appears in the top bar
AUTH-02	Register with empty fields	Click Create account without filling anything	"Name is required.", "Email is required.", "Password is required.", "Confirm password is required."
AUTH-03	Invalid email format	Enter "not-an-email" and submit	"Please enter a valid email address."
AUTH-04	Passwords do not match	Enter different passwords	"Passwords do not match."
AUTH-05	Weak password	Use password 12345 twice	"Password must be at least 8 characters long. This password is too common. Please choose a stronger one. Password cannot be only numbers."
AUTH-06	Duplicate email	Register again with an email that already has an account	"An account with this email already exists."
AUTH-07	Login with valid details	Log in with the email in capital letters and the correct password	Dashboard opens; localStorage key dayflow_token holds a 40-character token
AUTH-08	Login with wrong password	Correct email, wrong password	"Invalid email or password." with the red icon
AUTH-09	Login with empty fields	Click Log in without filling anything	"Email is required." and "Password is required."
AUTH-10	Logout	Click Log out	Login page; toast "You have been logged out."; token removed
AUTH-11	Stay logged in after refresh	Log in, open /profile, refresh the page	Still logged in; profile shows name and email

10.2 Access control and sessions

ID	Test case	Steps	Expected result
SEC-01	Protected pages when logged out	Open /dashboard, /tasks, /habits, /focus, /reports and /profile	Each redirects to /login; no protected content appears
SEC-02	Login page when logged in	While logged in, open /login and /register	Redirected to /dashboard
SEC-03	Fake token	Set localStorage dayflow_token to "fake"; open /dashboard	/login; the fake token is removed
SEC-04	Session ended elsewhere	Log out through the API with the same token; click Habits	/login with "Your session has expired. Please log in again."
SEC-05	Back button after logout	Log in, open /tasks, log out, press Back	Stays on /login; the Tasks page is not shown
SEC-06	Logout in another tab	Open DayFlow in two tabs; log out in one	The other tab goes to /login
SEC-07	Unknown address	Open /does-not-exist	404 page with a "Go to Dashboard" button

10.3 Tasks

ID	Test case	Steps	Expected result
TASK-01	Add a task	Tasks → + Add Task; title "Learn Java", priority High; Save	Toast "Task added."; the row shows High, Pending and today's date
TASK-02	Empty title	Open Add Task; click Save	"Title is required."; the modal stays open
TASK-03	Default values	Open Add Task	Priority Medium; due date = today; cursor in Task Title
TASK-04	No due date	Delete the date (Backspace) and save	The row shows "No due date"
TASK-05	Complete a task	Tick the task's checkbox	Toast "Task completed."; badge Completed; title crossed out
TASK-06	Search	Type "java" in the search box	Only tasks with "java" in the title or description remain
TASK-07	Filters	Priority = High; then Status = Pending	Only matching tasks are listed
TASK-08	No results	Search "zzz"; click Clear filters	"No tasks match your search or filters."; Clear filters shows all tasks again
TASK-09	Edit a task	Click Edit; change the title; Save	"Edit Task" opens filled in; toast "Task updated."; new title shown
TASK-10	Cancel editing	Edit, change the title, click Cancel (or press Escape)	Modal closes; nothing changes
TASK-11	Delete a task	Click Delete; Cancel; Delete again; confirm	Question 'Delete "<title>"? This cannot be undone.'; Cancel keeps the task; confirming shows "Task deleted."
TASK-12	Pagination	Have 11 tasks	"11 tasks", "Page 1 of 2"; Next shows page 2; Previous is disabled on page 1
TASK-13	Overdue task	A pending task with a past due date	Red "Overdue" badge
TASK-14	Task deleted elsewhere	Delete a task through the API, then tick it in the browser	"This task no longer exists. It may have been deleted."; the list refreshes
TASK-15	Phone layout	Open Tasks at phone width	Tasks are shown as cards; no sideways scrolling

10.4 Habits

ID	Test case	Steps	Expected result
HAB-01	Add a daily habit	+ Add Habit; name "Drink Water"; Save	Toast "Habit added."; badge "Every day"; "0 / 7 days"
HAB-02	Add a weekly habit	Frequency Weekly	"Days per week" appears (default 3); saved habit shows "3 days a week"
HAB-03	Empty name	Save without a name	"Name is required."
HAB-04	Tick a habit	Tick "Exercise"	Toast "Habit done for today. Keep it up!"; "1 / 3 days"; summary updates
HAB-05	Untick a habit	Untick it	Toast "Habit marked as not done today."
HAB-06	Weekly goal met	Weekly habit with 1 day a week; tick it	Badge "Weekly goal met"; counts as done
HAB-07	Ticks are saved	Refresh the page	Ticked habits are still ticked
HAB-08	Edit and delete	Edit a habit; delete another	Toasts "Habit updated." and "Habit deleted." (after confirmation)
HAB-09	Matches the dashboard	Tick 2 of 5 habits; open the Dashboard	Habits card shows 2/5

10.5 Focus Mode

ID	Test case	Steps	Expected result
FOC-01	No pending tasks	Open Focus Mode with no pending tasks	"You have no pending tasks…" and "Go to Tasks"
FOC-02	Setup screen	Open Focus Mode	First pending task selected; duration 25; clock 25:00
FOC-03	Invalid duration	Duration 0 (or 181); Start	"Duration must be between 1 and 180 minutes."
FOC-04	Start	Duration 1; Start	Task title shown; the clock counts down from 01:00
FOC-05	Pause and resume	Pause; wait; Resume	Clock stops and "Paused" appears; after Resume it continues
FOC-06	Stop	Stop; confirm	Question "Stop this focus session?"; toast "Focus session stopped."; setup screen
FOC-07	Complete	Start 1 minute; wait until 00:00	"Focus Session Completed!", "Duration: 1 minute", task name; toast "Focus session completed!"
FOC-08	Refresh while running	Refresh during a session	The same session continues counting down
FOC-09	Counts on the dashboard	Back to Dashboard after completing	Focus card shows "1 min"

10.6 Dashboard and Reports

ID	Test case	Steps	Expected result
DASH-01	Greeting	Log in	"Good morning / afternoon / evening, <name>!" and today's date
DASH-02	New account	Register and open the Dashboard	0/0, 0/0, 0 min, 0%; "No tasks due today."
DASH-03	Numbers match the data	10 tasks (7 done), 5 habits (4 done), 120 focus minutes	Cards 7/10, 4/5, 120 min and the score from /api/dashboard/productivity/
DASH-04	Tick a task on the Dashboard	Tick a task under Today's Tasks	Toast "Task completed. Nice work!"; cards, score and weekly bar update
DASH-05	Links	Click View all, Reports and your name	Open /tasks, /reports and /profile
REP-01	Weekly chart	Open Reports	One column for each day from Monday to today; later days show "—" in the table
REP-02	Chart labels	Look at the chart; hover a column	Only the best day has a label; hover shows the value and day
REP-03	Values	Compare with /api/dashboard/weekly-report/	Table values and data-score attributes match the API
REP-04	Summary tiles	Look at the tiles	Weekly average, best day and today's score are correct

10.7 Error handling

ID	Test case	Steps	Expected result
ERR-01	Network error	Stop the backend; save a task	"Can't reach the server. Check your internet connection and try again."
ERR-02	Server error	Make the API return 500	"Something went wrong on our side. Please try again in a moment."; no HTML or Django text
ERR-03	Loading fails	Stop the backend; open Tasks	"Could not load your tasks." with the reason and a Try again button that works once the server is back
ERR-04	No raw Django messages	Trigger form errors on every page	Never "This field may not be blank.", "Date has wrong format" or "Invalid pk"

10.8 API tests (Postman collection)

These API tests already exist in `postman/DayFlow.postman_collection.json`. Every request checks its status code, and the error cases also check the exact message.

Folder	Requests	What is tested
1. Auth	9	Register (and duplicate email, mismatch, weak password, missing fields), login (and wrong password), current user (and no token)
2. Tasks	16	Create (and blank title, invalid priority), list, get, PUT, PATCH complete, search, filters by status / priority / due date, sort, invalid filter, delete, deleted task gives 404
3. Habits	12	Create daily and weekly (and missing target), list, get, update, complete, no double counting, undo, delete
4. Focus	10	Start (and no task, already running), active, complete too early, cancel, complete after 61 s, history

Folder	Requests	What is tested
5. Dashboard	3	Dashboard numbers, productivity score (formula recalculated in the test), weekly report
6. Logout	2	Logout, and the old token is rejected
Total	52	106 checks; a full run takes about 70 seconds

11. How to Run the Application

DayFlow needs **two terminals**: one for the backend and one for the frontend. Keep both running while you use the app.

11.1 Terminal 1 — backend

```
cd DayFlow\backend
.\venv\Scripts\Activate.ps1
python manage.py runserver
```

11.2 Terminal 2 — frontend

```
cd DayFlow\frontend
npm run dev
```

11.3 Addresses

What	Address
DayFlow app	`http://localhost:5173`
API	`http://127.0.0.1:8000/api/`
Django admin	`http://127.0.0.1:8000/admin/`

11.4 First steps in the app

- Open http://localhost:5173 and click **Register**; after registering you are on the Dashboard.
- **Tasks**: add a few tasks for today and tick one.
- **Habits**: add a daily and a weekly habit and tick them.
- **Focus Mode**: pick a task, set 1 minute and press Start.
- Back on the **Dashboard** and in **Reports**, the numbers and today's score have updated.

11.5 Stop the application

Press **Ctrl+C** in each terminal. The data stays in the database for next time.

> **Development setup only.** DEBUG is on and SQLite is used, which is right for learning and testing. A public deployment would need DEBUG=False, a production database and web server, HTTPS and `npm run build` for the frontend.

12. How to Run Automation Tests

Start the backend and the frontend first (section 11). API tests need only the backend; Selenium tests need both.

12.1 API tests in the Postman app

- In Postman, click **Import** and choose `postman/DayFlow.postman_collection.json`.
- Right-click **DayFlow API** → **Run collection** → **Run DayFlow API**.
- Expected result: 52 requests and 106 checks, all passing, in about 70 seconds. "Complete Focus Session" waits 61 seconds on purpose.
- Run the whole collection in order: later requests use the token and ids saved by earlier ones.

12.2 API tests from the command line (Newman)

```
cd DayFlow
npx newman run postman\DayFlow.postman_collection.json                          # everything
npx newman run postman\DayFlow.postman_collection.json --folder "1. Auth"       # one folder
npx newman run postman\DayFlow.postman_collection.json --folder "1. Auth" --folder "2. Tasks"

# HTML report (open reports\api-report.html in Chrome). The ` at a line end continues the command.
npx -p newman -p newman-reporter-htmlextra newman run postman\DayFlow.postman_collection.json `
    -r cli,htmlextra --reporter-htmlextra-export reports\api-report.html

# JUnit XML report for CI tools such as Jenkins
npx newman run postman\DayFlow.postman_collection.json `
    -r cli,junit --reporter-junit-export reports\results.xml
```

Folders depend on each other: always include `"1. Auth"`, which creates the login token. Newman exits with code 0 when every check passes, and with a different code when something fails.

12.3 Selenium UI tests (after section 9 is set up)

```
cd DayFlow\automation
mvn test                                              # the whole testng.xml suite
mvn test -Dtest=AuthTests                             # one test class
mvn test -Dtest=AuthTests#invalidLoginShowsFriendlyMessage   # one test method
mvn test -Dheadless=true                              # without opening a Chrome window
```

Result	Where to find it
Pass / fail summary	At the end of the Maven output ("Tests run: ..., Failures: ...")
TestNG HTML report	automation/target/surefire-reports/index.html
Short e-mailable report	automation/target/surefire-reports/emailable-report.html

12.4 Test data

- Every API and UI test run registers new users with emails ending in @test.com, so tests never depend on each other's data.
- These accounts stay in the database. Remove them in the Django admin (Users → search "@test.com" → select → Delete), or reset the database as in section 5.6.

Appendix A. Known Limitations

- **Focus pause** is handled by the browser only. If the page is refreshed while paused, the timer continues from the server's remaining time, which kept counting during the pause.
- **Login tokens** do not expire until logout, and there is no limit on repeated login attempts. Both can be added later.
- **Past days in the weekly report** use each task's current status, so completing an overdue task later raises that day's score.
- **Habits page** loads up to 100 habits at once (the API's largest page).
- **Development settings**: DEBUG on and SQLite. See the note in section 11.5 before any public deployment.
- **Selenium automation** is planned and documented in section 9, but not yet part of the codebase.